

1. JPA vs Hibernate

JPA (Jakarta Persistence API) is a specification: a set of interfaces and annotations such as `EntityManager`, `@Entity`, `@Id` and `@OneToMany`. It describes how Java objects are mapped to relational tables but contains no implementation.

Hibernate ORM is the most widely used implementation of JPA. Spring Boot uses Hibernate by default when you add `spring-boot-starter-data-jpa`. Hibernate also offers features beyond the specification, for example `@BatchSize`, natural IDs, filters and statistics.

Coding against the JPA interfaces keeps your code portable, much like coding against `java.util.List` instead of `ArrayList`. You can still reach Hibernate-specific APIs by unwrapping the `EntityManager`: `entityManager.unwrap(Session.class)`.

Key building blocks:

- Entity: a class annotated with `@Entity` that maps to a table. It needs an identifier (`@Id`) and a no-argument constructor (it may be protected).
- EntityManager: the API to persist, find, merge and remove entities and to create queries. In Hibernate it corresponds to the Session.
- Persistence context: the set of managed entity instances inside an `EntityManager` (explained on page 3).
- JPQL: an object-oriented query language that works with entity names and fields instead of tables and columns, for example: `select b from Book b where b.author.name = :name`.

The schema can be generated by Hibernate (`spring.jpa.hibernate.ddl-auto=update`) but in production it is safer to manage it with migrations (Flyway or Liquibase) and use `ddl-auto=validate`.

2. Entity Lifecycle States

Every entity instance is in one of four states relative to a persistence context:

- Transient (new): created with new, not associated with any persistence context and without a database row. Hibernate does not track it.
- Managed (persistent): associated with a persistence context. Hibernate tracks every change and writes it to the database at flush time. Entities returned by find() or by queries are managed.
- Detached: was managed, but the persistence context has been closed or cleared, or detach() was called. Changes to a detached entity are not tracked.
- Removed: scheduled for deletion; the DELETE statement is executed at flush time.

Transitions:

- persist(entity): transient -> managed. The INSERT may be delayed until flush (except with IDENTITY ids, see page 10).
- merge(entity): copies the state of a detached (or transient) entity onto a managed instance and returns that managed instance. The object passed as an argument stays detached, a common source of bugs: always use the returned instance.
- remove(entity): managed -> removed.
- detach(entity), clear(), close(): managed -> detached.

In a typical Spring application, the persistence context lives as long as the transaction. When a @Transactional service method returns, its entities become detached.

3. Persistence Context, First-Level Cache and Dirty Checking

The persistence context works as a first-level cache. Within one persistence context, each database row is represented by at most one entity instance. Calling `find(Book.class, 1L)` twice runs only one `SELECT`; the second call returns the same object from the cache. The first-level cache is always on and cannot be disabled; it is scoped to a single persistence context, not shared between users.

Dirty checking: when an entity is loaded, Hibernate keeps a snapshot of its state. At flush time it compares every managed entity with its snapshot and generates `UPDATE` statements for changed entities. That is why you do not need to call `save()` after modifying a managed entity inside a transaction:

```
@Transactional
public void rename(Long id, String title) {
    Book book = bookRepository.findById(id).orElseThrow();
    book.setTitle(title);    // UPDATE is issued automatically at commit
}
```

Flushing synchronizes the persistence context with the database. With the default `FlushMode.AUTO`, Hibernate flushes before the transaction commits and before executing a query whose result could be affected by pending changes. `FlushMode.COMMIT` only flushes at commit.

Loading a very large number of entities in one persistence context uses a lot of memory because of the snapshots. For batch processing, flush and clear periodically, or use a read-only transaction, which skips snapshots.

4. Fetching Strategies: LAZY vs EAGER

The fetch type decides when an association is loaded.

- EAGER: the association is loaded together with the owning entity, immediately.
- LAZY: the association is loaded only when it is accessed for the first time. Until then Hibernate puts a proxy (for single-valued associations) or an uninitialized persistent collection in its place.

JPA defaults:

- @ManyToOne and @OneToOne: EAGER.
- @OneToMany and @ManyToMany: LAZY.

The recommended practice is to make every association LAZY, including @ManyToOne(fetch = FetchType.LAZY), and to fetch what a use case needs explicitly with a query. EAGER cannot be turned off per query: once mapped as EAGER, the association is loaded everywhere, often with extra queries that nobody asked for.

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "author_id")
private Author author;
```

Accessing a lazy association requires an open persistence context. Calling `book.getAuthor().getName()` after the transaction has ended throws `LazyInitializationException` (see page 7). Calling `getId()` on a lazy proxy does not trigger loading, because the id is already known.

5. The N+1 Select Problem

The N+1 select problem is the most common performance issue with ORM frameworks. It happens when code loads a list of N entities with one query and then accesses a lazy association of each entity, which triggers one additional query per entity: 1 + N queries in total.

```
List<Author> authors = authorRepository.findAll();           // 1 query
for (Author a : authors) {
    System.out.println(a.getBooks().size());                // +1 query per author
}
```

With 500 authors this code sends 501 SQL statements to the database. Each statement is fast, but the network round trips add up and the page becomes slow under load. The problem also appears with EAGER associations: loading N entities with a JPQL query and then fetching each eager association separately.

How to detect it:

- Log SQL statements during development (spring.jpa.show-sql=true, or better, logging.level.org.hibernate.SQL=debug) and look for the same SELECT repeated with different ids.
- Enable Hibernate statistics (spring.jpa.properties.hibernate.generate_statistics=true) and check the number of executed statements per request.
- Write tests that assert the number of queries.

The fixes are described on the next page.

6. Fixing N+1: JOIN FETCH, EntityGraph and Batch Fetching

1) JOIN FETCH in JPQL loads the association in the same query:

```
@Query("select a from Author a left join fetch a.books")  
List<Author> findAllWithBooks();
```

2) @EntityGraph declares which associations to fetch without writing JPQL:

```
@EntityGraph(attributePaths = "books")  
List<Author> findAll();
```

3) Batch fetching: set `hibernate.default_batch_fetch_size` (for example 50) or annotate the association with `@BatchSize(size = 50)`. Hibernate then initializes lazy associations of many entities with a single "where id in (...)" query, so N+1 queries become $1 + N/50$.

4) DTO projections: select only the columns you need into a record, for example `select new com.app.AuthorSummary(a.name, count(b)) from Author a left join a.books b group by a.name`.

Warning about pagination: combining JOIN FETCH of a collection with pagination makes Hibernate fetch all rows and paginate in memory, logging the warning `HHH90003004 "firstResult/maxResults specified with collection fetch; applying in memory"`. Fix it by first paging over the ids and then fetching the entities with their collections for those ids, or by using batch fetching.

Fetching two collections (bags) with JOIN FETCH in one query fails with `MultipleBagFetchException` and would produce a Cartesian product anyway; fetch them in separate queries.

7. LazyInitializationException and Open Session in View

LazyInitializationException ("could not initialize proxy - no Session") is thrown when a lazy association is accessed after the persistence context has been closed, typically in a controller or while Jackson serializes an entity after the @Transactional service method has returned.

Open Session in View (OSIV) keeps the persistence context open until the HTTP response is rendered, so lazy loading works in the web layer. Spring Boot enables it by default (spring.jpa.open-in-view=true) and logs a warning at startup. OSIV is considered an anti-pattern:

- The database connection is held for the whole request, including view rendering, which reduces throughput.
- Lazy loading from the view silently causes N+1 queries.
- Data access is no longer visible in the service layer.

Recommended approach: set spring.jpa.open-in-view=false and load everything a use case needs inside the transactional service method, using JOIN FETCH, @EntityGraph or DTO projections. Return DTOs rather than entities from controllers, so serialization cannot trigger lazy loading.

Calling Hibernate.initialize(entity.getBooks()) inside the transaction also works but is easy to forget. The setting hibernate.enable_lazy_load_no_trans hides the exception by opening a new session for every lazy load; it is strongly discouraged because it causes N+1 queries outside any transaction.

8. Cascade and orphanRemoval

In a bidirectional association, one side is the owner: the side with the foreign key, usually `@ManyToOne`. The other side declares `mappedBy` and is ignored when Hibernate writes the foreign key. Only changes on the owning side are persisted, so keep both sides in sync with helper methods:

```
public void addBook(Book book) {  
    books.add(book);  
    book.setAuthor(this);  
}
```

Cascade propagates entity operations from a parent to its children:

- `CascadeType.PERSIST`: persisting the parent also persists new children.
- `CascadeType.MERGE`: merging the parent also merges the children.
- `CascadeType.REMOVE`: deleting the parent deletes the children.
- `CascadeType.ALL`: all of the above plus `REFRESH` and `DETACH`.

Use cascading only from a parent to children that it owns completely (an `Order` and its `OrderLines`), never on `@ManyToOne` toward a shared entity. `CascadeType.REMOVE` on `@ManyToMany` is almost always a bug because it deletes entities that other rows still reference.

`orphanRemoval = true` deletes a child row when the child is removed from the parent's collection, for example `order.getLines().remove(line)`. `CascadeType.REMOVE` only acts when the parent itself is deleted; `orphanRemoval` also acts when the relationship is broken.

9. Optimistic and Pessimistic Locking

Concurrent transactions can overwrite each other's changes (the lost update problem). JPA offers two strategies.

Optimistic locking assumes conflicts are rare. Add a version field:

```
@Version
private Long version;
```

Every UPDATE includes "where id = ? and version = ?" and increments the version. If another transaction changed the row in the meantime, no row matches and Hibernate throws `OptimisticLockException` (in Spring, `ObjectOptimisticLockingFailureException`). The application can then reload the data and retry, or tell the user that the record was modified. Optimistic locking needs no database locks and scales well, which makes it the default choice for web applications.

Pessimistic locking locks the row in the database while reading it.

`LockModeType.PESSIMISTIC_WRITE` generates `SELECT ... FOR UPDATE`; other transactions that want to modify or lock the row wait until the lock holder commits.

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("select a from Account a where a.id = :id")
Account findForUpdate(Long id);
```

Use pessimistic locking when conflicts are frequent and retrying is expensive, for example decrementing stock of a popular product. Keep such transactions short, always lock rows in the same order to avoid deadlocks, and consider a lock timeout.

10. Identifier Generation Strategies

@GeneratedValue supports several strategies for primary keys:

- IDENTITY: uses an auto-increment column (SERIAL or IDENTITY in PostgreSQL, AUTO_INCREMENT in MySQL). The id is known only after the INSERT, so Hibernate must execute the INSERT immediately on persist(). This disables JDBC batch inserts, which hurts performance when inserting many rows.
- SEQUENCE: uses a database sequence. Hibernate fetches the id before the INSERT, so inserts can be batched. With allocationSize = 50 (the default), the pooled optimizer reserves a block of 50 ids per sequence call, which reduces round trips. The sequence INCREMENT BY must match allocationSize.
- TABLE: simulates a sequence with a table and row locks. It is slow and should be avoided.
- UUID: @GeneratedValue with a UUID field, or ids assigned by the application. UUIDs can be generated without the database and are unique across systems, but random UUIDs fragment B-tree indexes; time-ordered UUIDs (version 7) avoid most of that cost.

For PostgreSQL, SEQUENCE is usually the best choice for numeric keys.

To enable batching, set spring.jpa.properties.hibernate.jdbc.batch_size=50 and hibernate.order_inserts=true, which groups inserts of the same entity type together.

Implement equals() and hashCode() of entities carefully: do not base them on a generated id that is null before persist. Use a natural business key, or compare by id with a constant hashCode.